Day 1: Software Testing Fundamentals

Core concepts, QA vs QC, STLC vs SDLC, testing levels, and practical scenario design.

1. Day Learning Objectives

- [ ] Articulate why software testing is economically and operationally necessary
- [ ] Distinguish clearly between QA (process-oriented) and QC (product-oriented)
- [ ] Differentiate Verification (building it right) and Validation (building the right thing)
- [ ] Compare SDLC and STLC phases and explain why QA must shift left
- [ ] Classify human mistakes (error) vs code defects (bug) vs runtime failures
- [ ] Distinguish the four testing levels (Unit, Integration, System, Acceptance) and testing types
- [ ] Think like a tester by identifying positive, negative, boundary, and security test scenarios for a login module

2. Core Concepts & Practical Fundamentals

Topic 1: Introduction to Software Testing

Software testing is the systematic process of evaluating and verifying that a software application or system does what it is supposed to do, satisfies specified business requirements, and identifies defects before releasing to end customers.

Testing prevents catastrophic financial losses, security breaches, and reputational damage. Catching a defect in requirements or test planning costs 10x to 100x less than resolving an emergency outage in production.

What a Manual QA Engineer Does:
A manual QA engineer acts as the customer's advocate and the product's quality gatekeeper. Rather than just "clicking buttons," a QA engineer analyzes user stories, identifies hidden ambiguities, designs systematic test scenarios, executes tests with diverse data, logs clear and reproducible bug reports, and collaborates with developers and product managers to ensure software reliability.

Exhaustive Testing is Mathematically Impossible:
Testing all combinations of inputs, preconditions, timings, and environmental states would require infinite time and resources. Therefore, testing demonstrates the presence of defects, but cannot prove that software is completely defect-free. Professional QA relies on risk-based testing, equivalence partitioning, and boundary value analysis to maximize coverage within finite project constraints.

Real-World Workplace Examples:
• **[Fintech Mobile Banking App]:** A missing decimal rounding check in an international wire transfer module causes transfers of $10.005 to round up to $1,000, risking severe financial non-compliance and regulatory fines.
• **[Healthcare Patient Portal]:** An unvalidated session token allows Patient B to view Patient A's confidential blood test results by simply changing a URL patient ID query parameter.

Key Standard Terminology:
• **Software Testing:** The process of executing an application with the intent of finding defects and evaluating quality against requirements.
• **Manual QA Engineer:** A quality professional who manually executes test cases without automated scripts to evaluate software usability and functionality.
• **Exhaustive Testing Principle:** The principle that testing everything is impossible; testing must be prioritized based on risk.

Common Beginner Mistakes & Interview Traps:
& Believing that testing can prove software is 100% bug-free.
& Waiting until developers finish all coding before beginning test planning.
& Assuming that if code compiles without crash errors, it functions correctly according to business rules.

Topic 2: Essential QA Terminology

Clear technical communication is the trademark of a professional QA engineer. Here are the core definitions every tester must master:

• Quality Assurance (QA): Proactive, process-oriented activities focused on defect prevention through standards, reviews, and continuous process improvements.
• Quality Control (QC): Reactive, product-oriented activities focused on defect detection through test execution on built software.
• Verification: Static evaluation of work products (requirements, design, code) without execution to answer: "Are we building the product right?"
• Validation: Dynamic evaluation of executed software to answer: "Are we building the right product?"
• Error (Mistake): A human action made by an engineer (e.g. developer writes `>` instead of `>=`).
• Defect (Bug / Fault): The resulting flaw in code or documentation.
• Failure: The observable deviation during runtime execution when the defect is triggered.
• Test Scenario: A high-level description of what to test (e.g., "Verify user can log in with valid credentials").
• Test Case: A formal step-by-step document detailing how to test, including preconditions, test data, steps, and expected results.
• Expected Result vs Actual Result: The expected result is the required system behavior; the actual result is what the system did when

tested. A discrepancy indicates a defect.

Real-World Workplace Examples:

• **[Error vs Defect vs Failure Concrete Example]:** 1. Developer Error: A programmer forgets that leap years have 29 days in February.
2. Code Defect: The calendar algorithm sets `febDays = 28` unconditionally.
3. Runtime Failure: On Feb 29, 2028, users cannot book flight tickets because the date is rejected.

Key Standard Terminology:

• **Test Scenario:** A high-level "what to test" statement covering an end-to-end user goal or feature.
• **Test Case:** A formal set of conditions, steps, and expected outcomes to determine whether a feature satisfies requirements.
• **Actual Result:** The observable behavior or output of software during test execution.

Common Beginner Mistakes & Interview Traps:

& Using "bug", "error", and "failure" interchangeably during technical interviews.
& Confusing test scenarios (broad scope) with test cases (detailed steps and test data).

Topic 3: SDLC and STLC

Software Development Life Cycle (SDLC) defines the overall engineering stages:
1. Requirements Analysis: Gathering customer needs and defining functional specifications.
2. Design: Architecting system components, database schemas, and UI wireframes.
3. Development (Coding): Engineers write source code satisfying specifications.
4. Testing: QA executes test plans to detect defects and verify compliance.
5. Deployment: Release of software to staging and production environments.
6. Maintenance: Ongoing bug fixes, enhancements, and OS compatibility patches.

Software Testing Life Cycle (STLC) defines testing activities in detail:
1. Requirement Analysis: QA reviews user stories to identify testable requirements and ambiguities.
2. Test Planning: QA lead defines scope, schedule, resources, and test strategy.
3. Test Case Design: Testers write test scenarios, detailed test cases, and test data.
4. Test Environment Setup: Configuring servers, databases, and test client builds.
5. Test Execution: Running test cases, logging bugs in Jira, and tracking fixes.
6. Defect Reporting, Retesting & Regression: Verifying resolved bugs and checking unchanged areas.
7. Test Closure: Test summary report, metrics evaluation, and lessons learned.

Agile Overlap & Shift-Left Testing:
In traditional Waterfall, testing happened exclusively at the end. In modern Agile (Scrum/Kanban), SDLC and STLC phases overlap continuously in 2-week sprints. "Shift-Left" means moving testing activities as early as possible. When QA participates in Day 1 user story grooming, requirement bugs are caught before coding, saving up to 100x the cost of production fixes.

Real-World Workplace Examples:

• **[Shift-Left Example in Sprint Grooming]:** During user story grooming for a new discount coupon, QA asks: "Can a user apply both a 10% coupon and an employee discount simultaneously?" The Product Owner realizes they forgot to define this rule. Ambiguity is resolved in 5 minutes without writing buggy code.

Key Standard Terminology:

• **SDLC:** Software Development Life Cycle: the overall software delivery process from inception to maintenance.
• **STLC:** Software Testing Life Cycle: the structured sequence of testing activities within the SDLC.
• **Shift-Left Testing:** Involving QA in early development phases (grooming, requirements) to prevent defects.

Common Beginner Mistakes & Interview Traps:

& Thinking testing only begins after developers hand over compiled code.
& Viewing STLC as a linear, rigid waterfall process rather than iterative sprint cycles.

Topic 4: Testing Levels and Testing Types

Testing Levels represent WHERE in the software stack testing occurs:
1. Unit Testing: Testing individual functions, methods, or classes in isolation. Primarily conducted by developers using frameworks like Jest, JUnit, or PyTest.
   • Example: Testing that a `calculateSalesTax(price, rate)` function returns 10.00 for inputs 100 and 0.10.
2. Integration Testing: Testing data communication and interfaces between combined modules or services.
   • Example: Verifying that the frontend checkout form correctly sends JSON data to the backend payment microservice.
3. System Testing: End-to-end testing of the complete, integrated software build in an environment matching production.
   • Example: A tester opens the web portal, logs in, adds items to the cart, completes payment, and checks that a receipt email arrives.
4. Acceptance Testing (UAT): Testing by business stakeholders or actual end-users to confirm fitness for commercial release.
   • Example: Beta testers or client accounting teams verify monthly financial reports match business ledger expectations.

Testing Types represent WHAT aspect or PURPOSE of the software is evaluated:
• Functional Testing: Evaluates what the system does against functional specifications (e.g., successful login, invalid password handling, shopping cart tax computation).

• Non-Functional Testing: Evaluates how the system performs operational characteristics (e.g., Performance/Load testing under 10,000 concurrent users, Usability on mobile screens, Security penetration testing, Reliability during database failovers).

Change-Related Testing Types (Smoke, Sanity, Retesting, and Regression):
When software code changes (due to new feature additions, bug fixes, or dependency upgrades), QA runs targeted testing types:
• Smoke Testing (Build Verification): A fast, high-level test suite executed on every new build to verify that primary, critical functions work before QA accepts the build for deep testing. If the smoke test fails, the build is rejected.
• Sanity Testing: A focused, quick check performed after a minor bug fix or small code change to verify that the specific sub-module works without full end-to-end execution.
• Retesting: Testing specifically to verify that a previously reported defect has indeed been resolved and the exact bug no longer reproduces.
• Regression Testing: Testing unchanged parts of the software to ensure that new code changes, bug fixes, or enhancements have NOT introduced new bugs or broken existing functionality.

Note: A complete hands-on simulation of managing Regression Test Suites and Defect Triage is practiced in Day 3.

Real-World Workplace Examples:
• **[Functional vs Non-Functional in E-Commerce]:** Functional: User clicks "Place Order" and an order ID is generated in the database. Non-Functional: 5,000 users click "Place Order" simultaneously during Black Friday, and the page response time remains under 2 seconds without crashing.
• **[Retesting vs Regression Testing in Production]:** Developer resolves Bug #104: "Coupon code SUMMER20 does not apply discount."
1. Retesting: QA enters SUMMER20 and verifies the 20% discount is applied.
2. Regression Testing: QA places orders with other payment methods (Credit Card, PayPal, Gift Cards) and checks that regular prices and existing coupons still work without side-effects.

Key Standard Terminology:
• **Testing Levels:** Hierarchy of test stages: Unit !' Integration !' System !' Acceptance (UAT).
• **Functional Testing:** Verifying that system features work according to business specifications.
• **Non-Functional Testing:** Evaluating performance, security, usability, and stability attributes.
• **Smoke Testing:** Shallow test suite executed on a fresh build to verify critical paths before deeper testing.
• **Sanity Testing:** Focused test subset executed on a stable build to verify a specific bug fix or module update.
• **Retesting:** Executing the exact test cases that previously failed to verify that the defect has been corrected.
• **Regression Testing:** Testing unchanged areas of the application to guarantee that code changes did not break existing features.

Common Beginner Mistakes & Interview Traps:
& Confusing testing levels (e.g. System testing) with testing types (e.g. Regression or Functional testing).
& Thinking Retesting and Regression Testing are synonyms; retesting checks the bug fix, while regression checks untouched features.
& Believing non-functional testing is only for specialists; manual testers regularly test usability, responsiveness, and error recovery.

Topic 5: Think Like a Tester

Thinking like a tester means viewing software not just with the optimistic perspective of how it is supposed to work, but with the investigative curiosity of how it could fail under real-world conditions.

Key Mindsets:
• Positive Testing: Verifying that the system behaves correctly with valid inputs and normal user flows (Happy Path).
• Negative Testing: Verifying that the system gracefully handles invalid inputs, unauthorized actions, and boundary extremes without crashing or leaking sensitive data.
• Boundary & Edge Cases: Inspecting boundaries where logic transitions (e.g. passwords exactly at minimum 8 characters, or 7 characters, or 128 characters).
• Risk-Based Thinking: Prioritizing testing where failure causes the greatest business or security damage (e.g. financial transactions vs footer copyright text).

10 Common Login Scenarios to Consider:
1. Valid Login: Correct email + correct password redirects to dashboard.
2. Incorrect Password: Valid email + incorrect password shows generic error.
3. Unknown Email: Unregistered email shows generic error (never leak user existence).
4. Empty Fields: Submitting with empty email, empty password, or both.
5. Invalid Email Format: Inputs like `user@`, `user.com`, or spaces.
6. Password Masking: Characters are masked as dots/asterisks with toggle visibility option.
7. Rate Limiting / Lockout: 3-5 consecutive failed logins temporarily locks account.
8. Network Interruption: Submitting login while Wi-Fi drops or API is slow.
9. Appropriate Error Messages: Clear, secure error feedback without technical stack traces.
10. Session Expiry & Logout: Logging out invalidates session token so back button cannot re-enter.

Real-World Workplace Examples:

• **[Security Best Practice for Login Error Messages]:** Bad message: "Email not found" or "Password incorrect for user". (Allows hackers to enumerate valid email addresses).
Good message: "Invalid email or password. Please try again." (Protects user privacy).

Key Standard Terminology:

• **Positive Testing:** Testing software with valid inputs to confirm expected outcomes.
• **Negative Testing:** Testing software with invalid or unexpected data to verify error handling.
• **Risk-Based Testing:** Prioritizing test cases based on the probability and business impact of failure.

Common Beginner Mistakes & Interview Traps:

& Assuming every application requires identical rules; requirements and security specs must dictate behavior.
& Only testing the happy path and neglecting error messages, rate limiting, and network drop recovery.

3. Guided Practice Exercise

Guided Exercise: Deconstructing a Real-World Login Module

Analyze an enterprise login specification and systematically derive test scenarios for both valid and invalid pathways.

Feature Specification Under Test:

SPECIFICATION: Enterprise Authentication Module
1. Input fields: Email address (text) and Password (masked text).
2. Login button triggers authentication against identity database.
3. Valid credentials redirect user to /dashboard within 2 seconds.
4. Invalid credentials display: "Invalid email or password." (Never state whether email or password specifically was wrong for security).
5. Three consecutive failed attempts lock the account for 15 minutes and send an email alert.
6. "Remember Me" checkbox persists session token for 30 days.
7. Password reset link navigates to /forgot-password.

Recommended Execution Steps:

• Step 1: Identify Functional Capabilities (Happy path, error handling, security lockout, session persistence).
• Step 2: Define Positive Test Scenarios (Valid email + valid password, remember me checked, redirect verification).
• Step 3: Define Negative Test Scenarios (Wrong password, non-existent email, empty fields, SQL injection characters, 3-attempt lockout trigger).
• Step 4: Formulate Expected Behaviors for each scenario against security best practices.

4. Practical Workplace Assignment & Evaluation Rubric

Day 1 Practical Assignment: Comprehensive Test Scenario Discovery for Login System

Requirement: "A web application has email and password fields and a Login button. Registered users should be able to sign in."

Analyze the requirement and complete the test scenario workbook with at least 10 distinct scenarios across positive, negative, and input-validation categories. Include 2 clarification questions for the Business Analyst and identify your top 3 priority scenarios.

System Under Test Brief:

System Under Test: Web Application Login Module. Input fields: Email address, Password. Action: Login button. Target: Authenticate registered users and redirect to dashboard.

Submission Instructions:

• Identify at least 10 distinct test scenarios covering positive, negative, and input-validation cases.
• Document Scenario ID, Test Scenario description, Scenario Type, Purpose, and Priority (P1/P2/P3).
• Write two clarification questions you would ask a Business Analyst (BA) before finalizing test cases.
• Select the three highest-priority scenarios and provide a concise justification for why they come first.
• Click "Submit Assignment" to run the evaluation engine.

Deterministic Evaluation Rubric:

' **[Weight 30%] Minimum 10 Test Scenarios [Requires at least 10]:** Learner documents at least 10 meaningful, distinct test scenarios.
' **[Weight 35%] Positive, Negative & Validation Coverage:** Includes valid flows, negative authentication flows, and field validation.
' **[Weight 35%] Clear Purpose & Priority Justification:** Detailed purpose for each scenario and valid BA clarification questions.

5. Industry Interview Questions & Model Answers

Q1 [Foundational]: 1. What is software testing?

Model Answer:

Software testing is the systematic process of executing an application or system to verify that it satisfies specified business requirements, identify defects before release, and evaluate overall software quality and operational reliability.

Interviewer Follow-Up: Can software testing prove that a system is 100% bug-free?

Avoid this mistake: Defining testing merely as "clicking around to see if it breaks".

Q2 [Foundational]: 2. Why is software testing necessary?

Model Answer:

Software testing is essential because software defects can cause catastrophic financial losses, data leaks, security vulnerabilities, and brand damage. Economically, catching a bug during requirement grooming or testing costs 10x to 100x less than resolving an outage in production.

Avoid this mistake: Focusing only on cosmetic bugs rather than business risk and economics.

Q3 [Difference-Based]: 3. What is the difference between QA and QC?

Model Answer:
Quality Assurance (QA) is proactive and process-oriented; it focuses on defect prevention through sprint grooming, standardizing test processes, and code review criteria. Quality Control (QC) is reactive and product-oriented; it focuses on defect detection by executing test cases on actual software builds to inspect results.

Interviewer Follow-Up: Name a QA activity you perform before code is written.

Avoid this mistake: Saying QA is manual testing and QC is automated testing.

Q4 [Difference-Based]: 4. What is the difference between verification and validation?

Model Answer:
Verification evaluates documents, plans, and code statically without execution to answer: "Are we building the product right?" (e.g. reviewing user story acceptance criteria). Validation evaluates the executed software dynamically to answer: "Are we building the right product?" (e.g. submitting a payment transaction in staging to verify money is deducted and an order is created).

Avoid this mistake: Reversing the questions "building it right" vs "building the right product".

Q5 [Terminology]: 5. Explain error, defect, and failure with a clear example.

Model Answer:
An Error is a human mistake made by an engineer (e.g. typing > instead of >= in code). A Defect (bug) is the manifestation of that mistake in the software code or specification. A Failure is the observable runtime deviation from expected behavior when the software executes that flawed code (e.g. an account with exactly $100 is blocked from purchasing a $100 item).

Avoid this mistake: Calling the code flaw a failure before the program is executed.

Q6 [Process]: 6. What are SDLC and STLC?

Model Answer:
SDLC (Software Development Life Cycle) encompasses the complete engineering journey: Requirements, Design, Development, Testing, Deployment, and Maintenance. STLC (Software Testing Life Cycle) is the specialized subset of activities focused on quality: Requirement Analysis, Test Planning, Test Case Design, Environment Setup, Test Execution, Defect Reporting, and Test Closure.

Avoid this mistake: Thinking STLC only begins after the development phase is completely finished.

Q7 [Process]: 7. At what stage should QA become involved in a project?

Model Answer:
QA should become involved as early as possible—specifically during the Requirements Analysis phase (known as "Shift-Left"). Involving QA early allows testers to identify ambiguous requirements, missing edge cases, and architectural constraints before code is written, which is the most cost-effective stage to resolve flaws.

Avoid this mistake: Saying QA should wait until developers deploy code to the test server.

Q8 [Difference-Based]: 8. What is the difference between functional and non-functional testing?

Model Answer:
Functional testing evaluates WHAT the system does against functional requirements (e.g. successful login, payment calculation, password reset). Non-functional testing evaluates HOW the system performs operational characteristics (e.g. performance under high load, security vulnerabilities, usability, and reliability during network drops).

Avoid this mistake: Thinking security testing is functional just because it involves passwords.

Q9 [Concept]: 9. Explain the four testing levels.

Model Answer:
The four testing levels are: 1. Unit Testing (testing individual functions or methods in isolation, usually by developers), 2. Integration Testing (testing communication interfaces and data exchange between combined modules or APIs), 3. System Testing (end-to-end testing of the complete integrated application), and 4. Acceptance Testing (UAT - verifying that the product meets business expectations and user workflows).

Avoid this mistake: Confusing testing levels (Unit, Integration, System, UAT) with testing types (Smoke, Regression, Functional).

Q10 [Practical Approach]: 10. How would you start testing a login page?

Model Answer:
I start by analyzing the requirements to understand expected behaviors (session timeouts, lockout policies). Then I formulate a test plan covering: 1. Positive scenarios (valid login, remember me, redirection), 2. Negative scenarios (wrong password, unknown email, empty inputs, bad email formats), 3. Boundary scenarios (minimum and maximum password lengths), 4. Security scenarios (password masking, SQL injection attempts, rate limiting after 3-5 failed attempts), and 5. Non-functional checks (load time, responsive UI on mobile, and network drop recovery).

Avoid this mistake: Only listing 1 or 2 happy-path scenarios and forgetting security and error handling.

Q11 [Scenario Investigation]: Scenario Investigation: "A customer makes a payment. The money is deducted, but no order confirmation appears. How would you investigate this issue as a QA engineer?"

Model Answer:
Investigation Approach:
1. Check Expected Behavior: Review payment gateway flow specs to confirm if order confirmation is created before or after gateway callback webhook.
2. Reproduce in Test Environment: Attempt to reproduce using staging payment sandbox credentials with network logging open.
3. Gather Evidence & Inspect Logs: Inspect browser DevTools (Network tab for failed POST requests or HTTP 500), backend API logs, and database tables (`payments` vs `orders`) to check if the payment record was captured without order creation.
4. Check Third-Party Payment Gateway Dashboard: Verify transaction status in the gateway (Stripe/Razorpay) to confirm if the payment succeeded, is pending, or failed on the bank's end.
5. Document Clear Bug Report: Log a Critical defect with full evidence: transaction ID, timestamp, user ID, API response payloads, screenshots, and steps to reproduce, without assuming the root cause before developers inspect the webhook listener.

Avoid this mistake: Blaming the bank or user without checking backend API logs and webhook listeners.

6. Key Takeaways & Fast Revision Notes

Core Takeaways:

- Testing demonstrates the presence of defects, never their total absence.
- QA prevents bugs early; QC detects bugs in executed software.
- Verification is static inspection; Validation is dynamic execution.
- Human error leads to code defects, which produce observable runtime failures.
- Early QA involvement (Shift-Left) saves up to 100x the cost of production defect resolution.

Cheat Sheet / Revision Notes:

- 7 Testing Principles: 1. Testing shows presence of defects, 2. Exhaustive testing impossible, 3. Early testing, 4. Defect clustering, 5. Pesticide paradox, 6. Testing is context dependent, 7. Absence-of-errors fallacy.
- Test Levels: Unit (dev), Integration (interfaces), System (end-to-end), Acceptance (UAT).
- Test Types: Functional (what system does) vs Non-functional (performance, security, usability).
- Login Checklist: Valid credentials, invalid password, unregistered email, empty inputs, invalid format, password masking, lockout/rate limiting, network timeout, generic security error messages.